

Lean 4 Formalization — Propositions 3.1 and 3.3

Michel Monfette, 2026

mycmon@gmail.com

Companion document to `MonfetteGoldbach.lean`

Overview

This document presents the Lean 4 / Mathlib formalization of the two proved combinatorial propositions from the paper
"Partition of Goldbach Pairs via $(\mathbb{Z}/30\mathbb{Z})^\star$ ".

Element	Status
Prop 3.1 — exact tunnel count table	✓ Proved
Prop 3.3a — disjointness of tunnels	✓ Proved
Prop 3.3b — exhaustive coverage	✓ Proved
Prop 3.3 — full equality	✓ Proved
Lemma 3.4 — involution α	✓ Proved
Lemma <code>prime_gt5_mod_mem_R30</code>	✓ Proved (resolved)
Corollary <code>admissible_le_eight</code>	✓ Proved (resolved)
Synthesis theorem	✓ Proved

Zero sorry in the final file.

Section 1: Definitions

R30 — the group $(\mathbb{Z}/30\mathbb{Z})^\star$

```
def R30 : Finset ℕ := {1, 7, 11, 13, 17, 19, 23, 29}
```

Every prime `p > 5` satisfies `p % 30 ∈ R30`.

Basic properties proved by `decide`:

```
lemma R30_card : R30.card = 8 := by decide

lemma R30_coprime (a : ℕ) (ha : a ∈ R30) : Nat.Coprime a 30 := by
  simp [mem_R30_iff] at ha
  rcases ha with rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl <|> decide
```

admissibleTunnels — tunnels for N

```
def admissibleTunnels (N : ℕ) : Finset (ℕ × ℕ) :=
  (R30 ×s R30).filter (fun ab => (ab.1 + ab.2) % 30 = N % 30)
```

Ordered pairs (a, b) with $a, b \in \mathbb{R}_{30}$ and $a + b \equiv N \pmod{30}$.

Design choice: we work with *ordered* pairs (no constraint $a \leq b$).

Each Goldbach pair (p, q) belongs to exactly the tunnel $(p \% 30, q \% 30)$, with no ambiguity of ordering.

tunnelCountOrd — the cardinality table

```
def tunnelCountOrd (r : ℕ) : ℕ :=
  match r with
  | 0      => 8
  | 6|12|18|24 => 6
  | 10|20   => 4
  | _      => 3  -- all other even residues
```

goldPairsGt5 and goldTunnel

```
def goldPairsGt5 (N : ℕ) : Finset (ℕ × ℕ) :=
  ((Finset.range N) ×s (Finset.range N)).filter
    (fun pq =>
      7 ≤ pq.1 ∧ 7 ≤ pq.2 ∧
      Nat.Prime pq.1 ∧ Nat.Prime pq.2 ∧
      pq.1 + pq.2 = N)

def goldTunnel (N a b : ℕ) : Finset (ℕ × ℕ) :=
  (goldPairsGt5 N).filter (fun pq => pq.1 % 30 = a ∧ pq.2 % 30 = b)
```

Classification G1 / G2 / G3

```
inductive TunnelType | G1 | G2 | G3 deriving DecidableEq, Repr

def classifyTunnel (a b : ℕ) : TunnelType :=
  if a = b      then .G3
  else if a+b=30 then .G1
  else          .G2
```

Section 2: Proposition 3.1

Statement: for every even N , the number of admissible tunnels is determined entirely by $N \% 30$.

Key lemma: dependence only on $N \% 30$

```
lemma admissibleTunnels_mod (N : ℕ) :  
  admissibleTunnels N = admissibleTunnels (N % 30) := by  
  simp only [admissibleTunnels]  
  congr 1; ext (a, b)  
  simp only [Finset.mem_filter]  
  constructor  
  · rintro ⟨h, hmod⟩; exact ⟨h, by omega⟩  
  · rintro ⟨h, hmod⟩; exact ⟨h, by omega⟩
```

Theorem (Prop 3.1)

```
theorem prop3_1 (N : ℕ) (hN : N % 2 = 0) :  
  (admissibleTunnels N).card = tunnelCountOrd (N % 30) := by  
  rw [admissibleTunnels_mod]  
  have hlt : N % 30 < 30 := Nat.mod_lt N (by norm_num)  
  interval_cases (N % 30) <=>  
    simp [admissibleTunnels, R30, tunnelCountOrd] <=>  
    decide
```

Proof strategy: reduce to `admissibleTunnels (N % 30)`, then `interval_cases` splits into the 15 even values of `[0, 29]`, and `decide` computes the exact cardinality in each case.

Corollaries

```
-- There are always at least 3 admissible tunnels  
corollary admissible_pos (N : ℕ) (hN : N % 2 = 0) :  
  0 < (admissibleTunnels N).card := by  
  rw [prop3_1 N hN]  
  have hlt : N % 30 < 30 := Nat.mod_lt N (by norm_num)  
  interval_cases (N % 30) <=> simp [tunnelCountOrd]  
  
-- The cardinality is at most 8 (achieved for  $N \equiv 0 \pmod{30}$ )  
corollary admissible_le_eight (N : ℕ) (hN : N % 2 = 0) :  
  (admissibleTunnels N).card ≤ 8 := by  
  rw [prop3_1 N hN]  
  have hlt : N % 30 < 30 := Nat.mod_lt N (by norm_num)  
  interval_cases (N % 30) <=> simp [tunnelCountOrd]
```

Section 3: Fundamental Lemma — residue of a prime > 5

This lemma is the key to Proposition 3.3. It was previously marked `sorry`. It is now **completely proved**.

Why `omega` alone failed

`omega` reasons about linear inequalities. It cannot deduce that $p \% 30 \in \{1, 7, \dots, 29\}$ from $p\%2 \neq 0$, $p\%3 \neq 0$, $p\%5 \neq 0$ alone, because this reasoning crosses three modular conditions simultaneously (Chinese Remainder Theorem). `omega` has no knowledge of CRT.

The solution: three relay facts + interval_cases

```
lemma prime_gt5_mod_mem_R30 (p : ℕ) (hp : Nat.Prime p) (h5 : 5 < p) :
  p % 30 ∈ R30 := by
  -- Step 1: a prime p > 5 cannot be divisible by 2, 3, or 5
  have h2 : ¬ 2 ∣ p := fun h =>
    absurd (hp.eq_one_or_self_of_dvd 2 h) (by omega)
  have h3 : ¬ 3 ∣ p := fun h =>
    absurd (hp.eq_one_or_self_of_dvd 3 h) (by omega)
  have h5d : ¬ 5 ∣ p := fun h =>
    absurd (hp.eq_one_or_self_of_dvd 5 h) (by omega)
  -- Step 2: translate to residue conditions
  have h2m : p % 2 ≠ 0 := fun h => h2 (Nat.dvd_of_mod_eq_zero h)
  have h3m : p % 3 ≠ 0 := fun h => h3 (Nat.dvd_of_mod_eq_zero h)
  have h5m : p % 5 ≠ 0 := fun h => h5d (Nat.dvd_of_mod_eq_zero h)
  -- Step 3: bound on p % 30
  have hlt : p % 30 < 30 := Nat.mod_lt p (by norm_num)
  -- Step 4 (KEY): link p%30 to p%2, p%3, p%5 via omega
  -- omega can prove these because 2|30, 3|30, 5|30 (linear divisibility)
  have hr2 : p % 30 % 2 = p % 2 := by omega
  have hr3 : p % 30 % 3 = p % 3 := by omega
  have hr5 : p % 30 % 5 = p % 5 := by omega
  -- Step 5: interval_cases enumerates all 30 cases for p%30
  -- In each case r ∈ {0..29}, omega checks:
  --   r%2=0 eliminates 0,2,4,6,8,10,12,14,16,18,20,22,24,26,28
  --   r%3=0 eliminates 3,9,15,21,27
  --   r%5=0 eliminates 5,25
  --   the 8 survivors are exactly R30
  simp only [mem_R30_iff]
  interval_cases (p % 30) <|> omega
```

Detailed mechanism

Residue r	Eliminated by	Reason
0,2,4,6,8,10,12,14,16,18,20,22,24,26,28	<code>h2m</code>	<code>r % 2 = 0</code>
3, 9, 15, 21, 27	<code>h3m</code>	<code>r % 3 = 0</code>

Residue r	Eliminated by	Reason
5, 25	<code>h5m</code>	<code>r % 5 = 0</code>
1,7,11,13,17,19,23,29	—	survivors = R30 ✓

Section 4: Proposition 3.3

Linking lemmas

```
-- Every pair in goldPairsGt5 has its residues in R30
lemma goldPairs_mod_mem_R30 (N p q : ℕ) (h : (p,q) ∈ goldPairsGt5 N) :
  p % 30 ∈ R30 ∧ q % 30 ∈ R30

-- Every pair belongs to its own tunnel
lemma mem_own_tunnel (N p q : ℕ) (h : (p,q) ∈ goldPairsGt5 N) :
  (p,q) ∈ goldTunnel N (p % 30) (q % 30)

-- The tunnel (p%30, q%30) is admissible for N
lemma own_tunnel_admissible (N p q : ℕ) (h : (p,q) ∈ goldPairsGt5 N) :
  (p % 30, q % 30) ∈ admissibleTunnels N
```

Theorem 3.3a — Disjointness

```
theorem prop3_3_disjoint (N : ℕ) :
  (admissibleTunnels N).PairwiseDisjoint
    (fun t => goldTunnel N t.1 t.2) := by
  intro ⟨a,b⟩ _ ⟨c,d⟩ _ hne
  rw [Finset.disjoint_left]
  intro ⟨p,q⟩ hab hcd
  apply hne
  simp [goldTunnel, Finset.mem_filter] at hab hcd
  ext <=> simp [← hab.2.1, ← hcd.2.1, ← hab.2.2, ← hcd.2.2]
```

Key idea: if (p, q) belonged to two tunnels (a, b) and (c, d) , then $p\%30 = a = c$ and $q\%30 = b = d$, contradicting $(a, b) \neq (c, d)$.

Theorem 3.3b — Coverage

```
theorem prop3_3_cover (N : ℕ) :
  goldPairsGt5 N ⊆
    (admissibleTunnels N).biUnion (fun t => goldTunnel N t.1 t.2) := by
  intro ⟨p,q⟩ hpq
  simp only [Finset.mem_biUnion]
  exact ⟨(p % 30, q % 30),
    own_tunnel_admissible N p q hpq,
    mem_own_tunnel N p q hpq⟩
```

Theorem 3.3 — Full Equality

```
theorem prop3_3 (N : ℕ) :
  goldPairsGt5 N =
    (admissibleTunnels N).biUnion (fun t => goldTunnel N t.1 t.2) :=
  Finset.Subset.antisymm (prop3_3_cover N) (prop3_3_sub N)
```

Section 5: Lemma 3.4 — The Involution α

```
def alpha (r : ℕ) : ℕ := 11 * r % 30

theorem alpha_involution (r : ℕ) (hr : r ∈ R30) :
  alpha (alpha r) = r := by
  simp [mem_R30_iff] at hr
  rcases hr with rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl <=>
    simp [alpha]

theorem alpha_mem_R30 (r : ℕ) (hr : r ∈ R30) : alpha r ∈ R30 := by
  simp [mem_R30_iff] at hr
  rcases hr with rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl <=>
    simp [alpha, mem_R30_iff]

theorem alpha_orbits_explicit :
  ∀ r ∈ R30, ({r, alpha r} : Finset ℕ) ∈
    ({({1,11} : Finset ℕ), {7,17}, {13,23}, {19,29}} : Finset (Finset ℕ)) := by
  intro r hr
  simp [mem_R30_iff] at hr
  rcases hr with rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl <=>
    simp [alpha]
```

The four orbits `{1,11}`, `{7,17}`, `{13,23}`, `{19,29}` are proved exhaustively by case analysis on all 8 elements of R30.

Section 6: Synthesis Theorem

```
theorem monfette_partition_theorem (N : ℕ) (hN : N % 2 = 0) :
  -- (i) disjoint exhaustive partition
  goldPairsGt5 N =
    (admissibleTunnels N).biUnion (fun t => goldTunnel N t.1 t.2) ∧
  -- (ii) tunnels pairwise disjoint
  (admissibleTunnels N).PairwiseDisjoint
    (fun t => goldTunnel N t.1 t.2) ∧
  -- (iii) exact cardinality table
  (admissibleTunnels N).card = tunnelCountOrd (N % 30) :=
  (prop3_3 N, prop3_3_disjoint N, prop3_1 N hN)
```

Final Summary

Theorem	Main tactics	Status
<code>R30_card</code>	<code>decide</code>	✓
<code>R30_coprime</code>	<code>rcases</code> + <code>decide</code>	✓
<code>admissibleTunnels_mod</code>	<code>congr</code> , <code>omega</code>	✓
<code>prop3_1</code>	<code>interval_cases</code> , <code>decide</code>	✓
<code>admissible_pos</code>	<code>interval_cases</code>	✓
<code>admissible_le_eight</code>	<code>prop3_1</code> + <code>interval_cases</code>	✓
<code>prime_gt5_mod_mem_R30</code>	<code>interval_cases</code> + <code>omega</code>	✓
<code>prop3_3_disjoint</code>	<code>Finset.disjoint_left</code>	✓
<code>prop3_3_cover</code>	<code>Finset.mem_biUnion</code>	✓
<code>prop3_3</code>	<code>antisymm</code>	✓
<code>alpha_involution</code>	<code>rcases</code> + <code>simp</code>	✓
<code>alpha_orbits_explicit</code>	<code>rcases</code> + <code>simp</code>	✓
<code>monfette_partition_theorem</code>	combination	✓

13 theorems proved. Zero sorry.

How to Run in Lean 4

```
# Create a Lake project
lake new MonfetteGoldbach
cd MonfetteGoldbach

# Add to lakefile.lean:
# require mathlib from git
#   "https://github.com/leanprover-community/mathlib4.git"

# Copy MonfetteGoldbach.lean into the project folder
lake update
lake build
```

End of document — Version 3.0 (zero sorry)